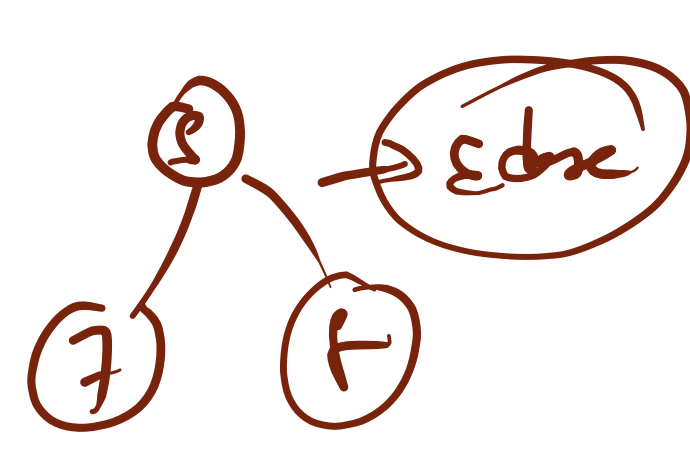
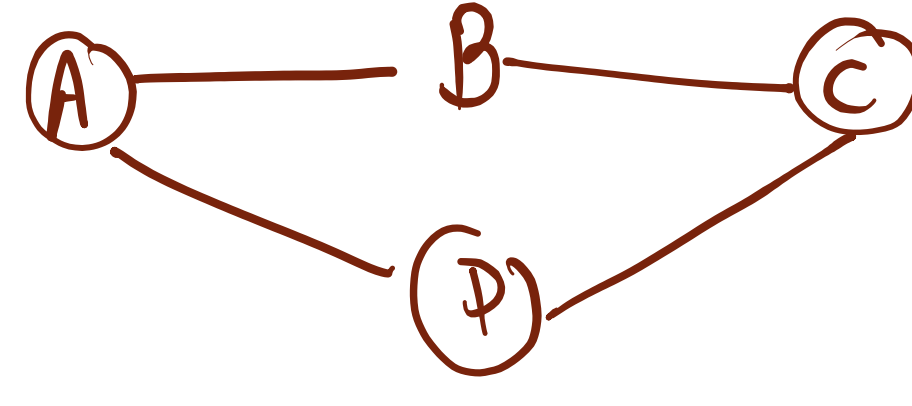


GraphTree $\rightarrow$ GraphGraph $\rightarrow$ Tree X

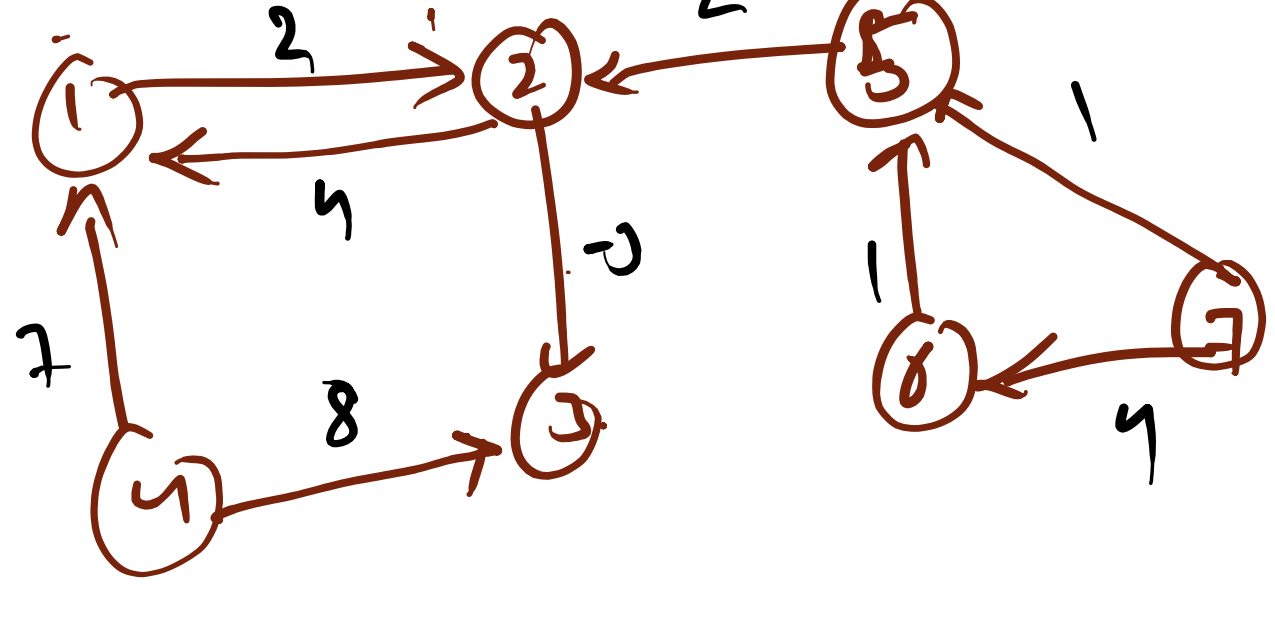
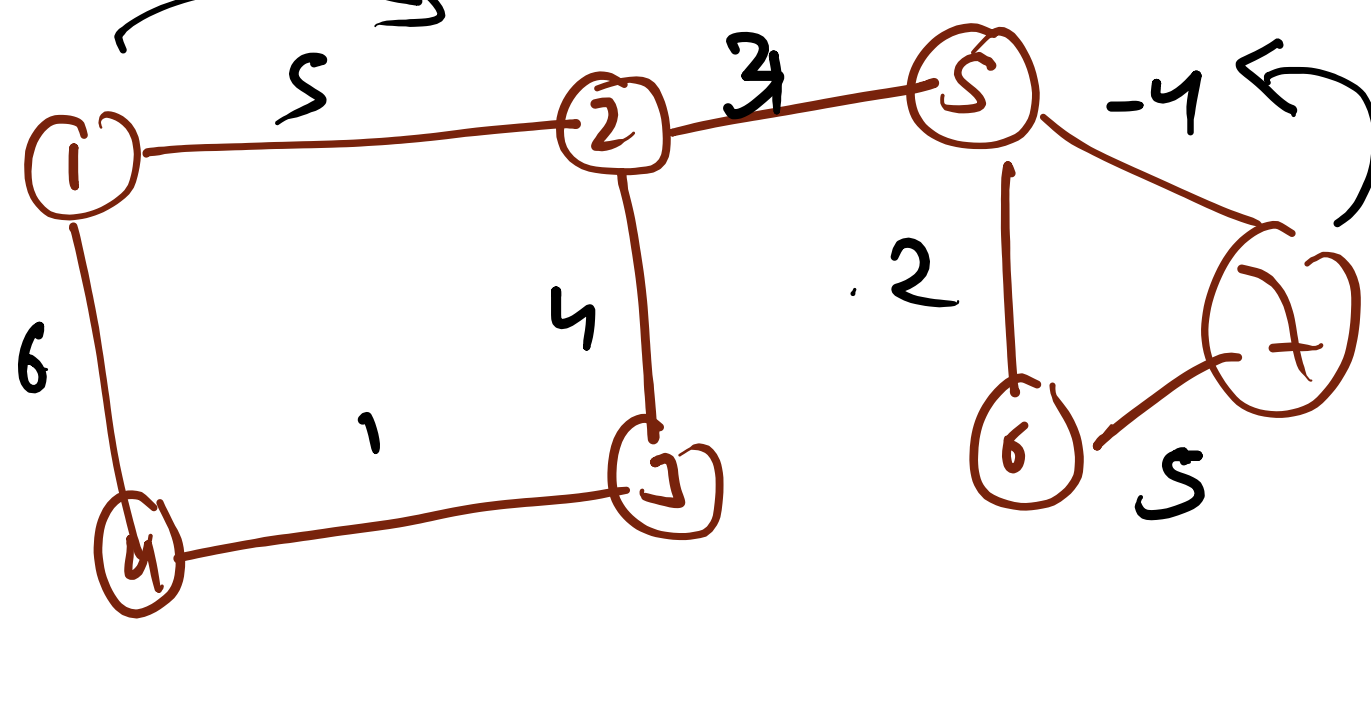
Finite set Edge and Vtx

node
vtx

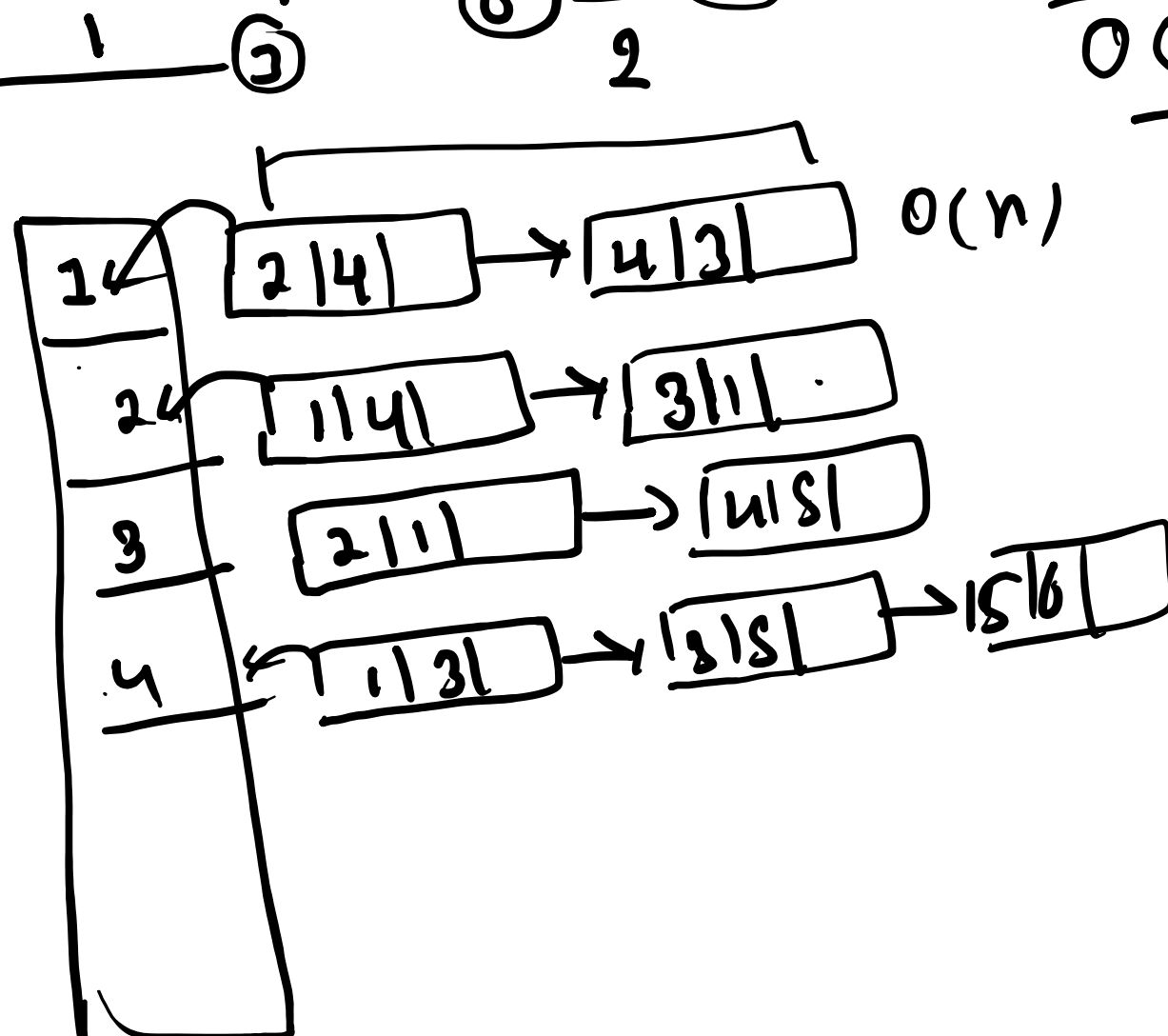
① Graph



- ① directed graph $\rightarrow$ weighted
② undirected graph $\rightarrow$ unweighted



① Adj-List



① Adj matrix

	1	2	3	4	5	6	7
1	0	1	0	0	0	0	0
2	1	0	1	0	0	0	0
3	0	1	0	0	1	0	0
4	0	0	0	1	0	1	0
5	0	0	1	0	0	0	1
6	0	0	0	1	0	1	0
7	0	0	0	0	1	0	1

7x7

In Map<int, int>

	1K	2K	3K	4K	5K	6K	7K
1	1K						
2		2K					
3			3K				
4				4K			
5					5K		
6						6K	
7							7K

1	4
4	3

1K

1	4
3	1

2K

2	1
4	5

3K

1	3
3	5
5	6

4K

4	6
7	2
6	7

5K

5	7
7	4

6K

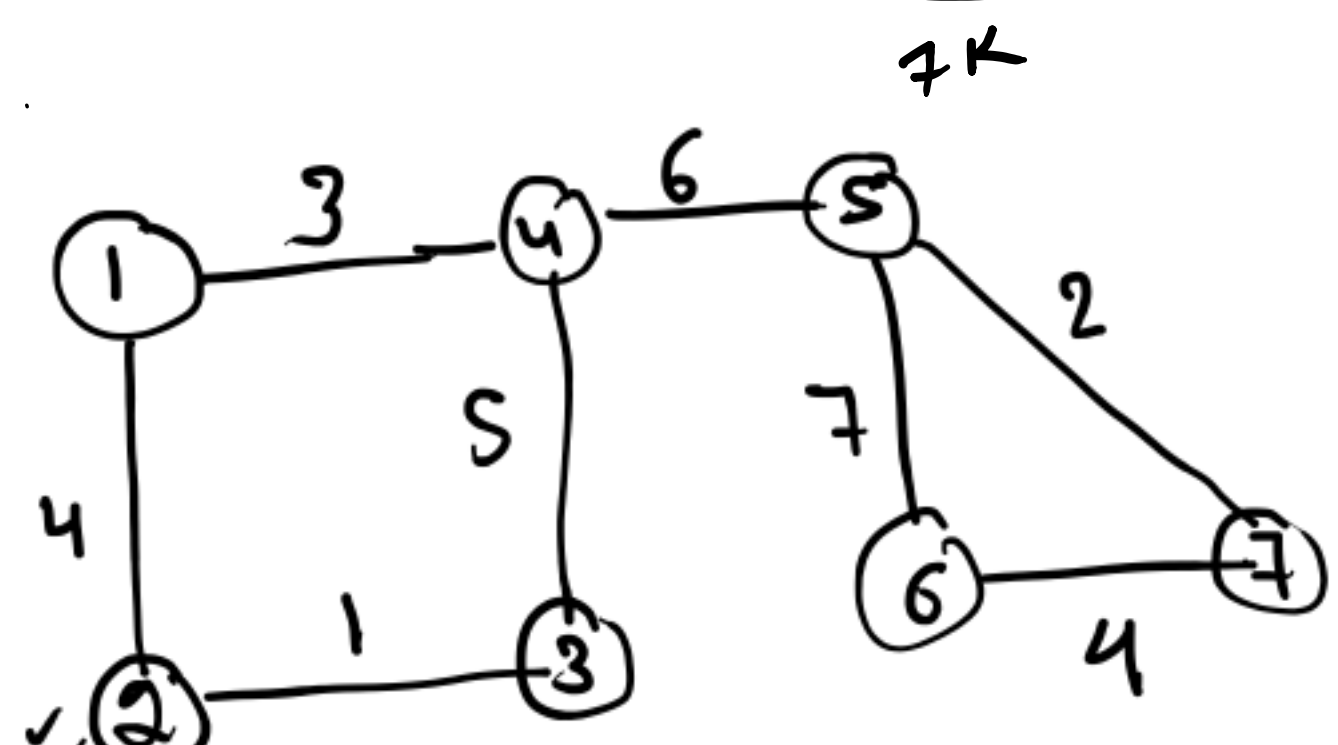
5	2
6	4

7K

```

public boolean haspath(int src, int des) {
    if (src == des) {
        return true;
    }
    for (int nbrs : map.get(src).keySet()) {
        boolean ans = haspath(nbrs, des);
        if (ans) {
            return true;
        }
    }
    return false;
}

```



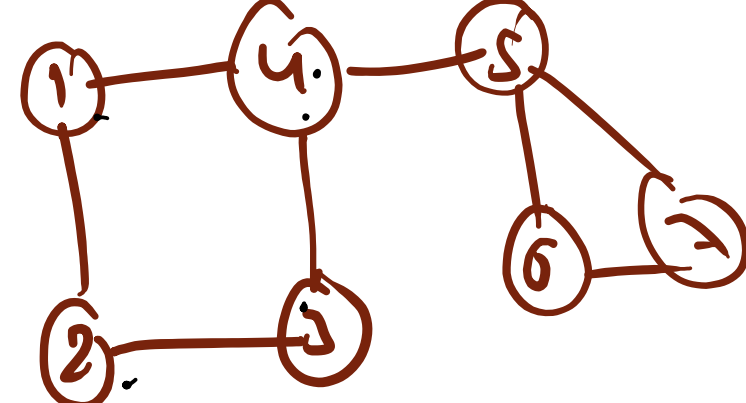
	src=1 des=6
(1, 3) (1, 1)	src=2 des=6
(2, 4) (2, 1)	src=1 des=6
(1, 6)	

map

```

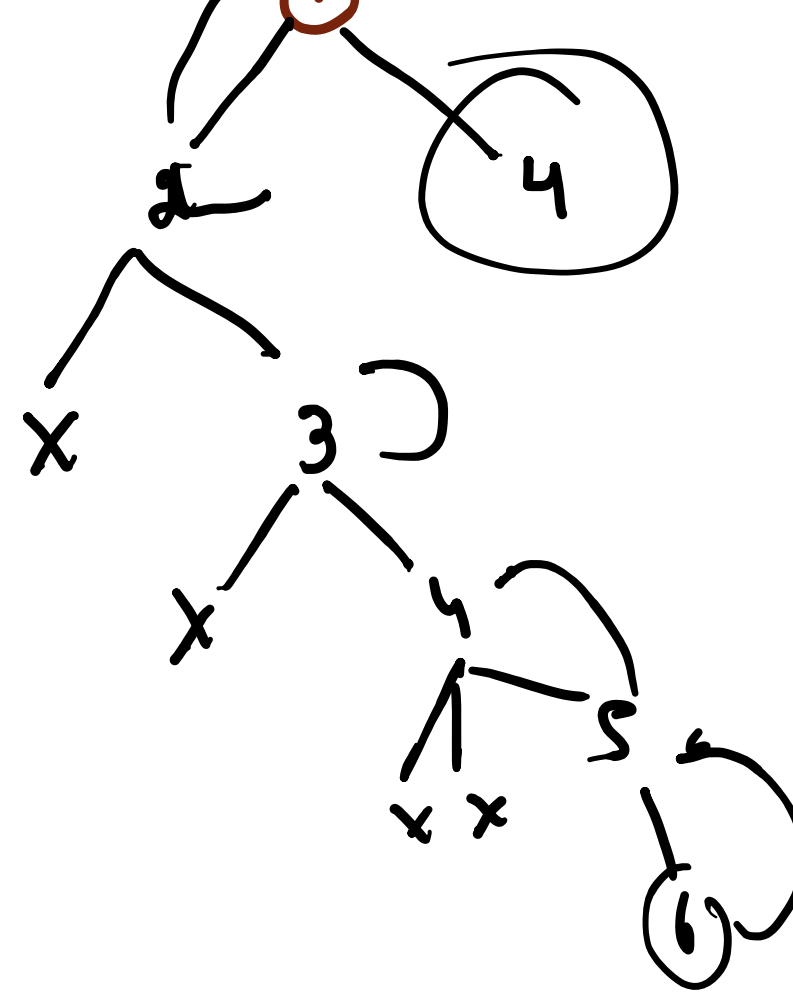
public boolean haspath(int src, int des, HashSet<Integer> visited) {
    if (src == des) {
        return true;
    }
    visited.add(src);
    for (int nbrs : map.get(src).keySet()) {
        if (!visited.contains(nbrs)) {
            boolean ans = haspath(nbrs, des, visited);
            if (ans) {
                return true;
            }
        }
    }
    return false;
}

```



1, 2, 3, 4, 5

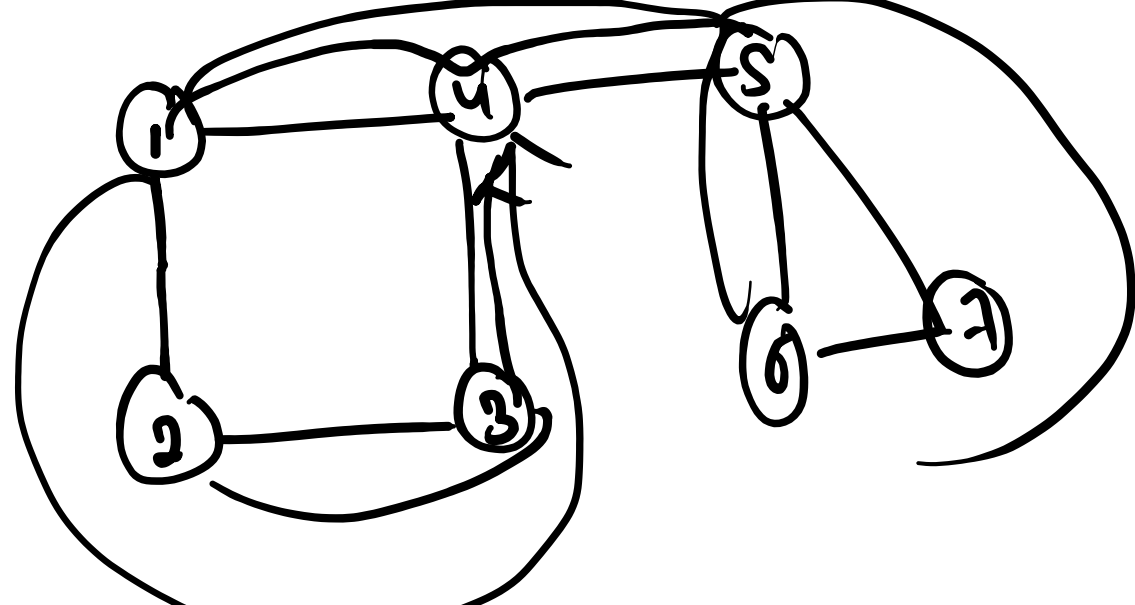
1, 2, 3, 4, 5



```

public void Printpath(int src, int des, HashSet<Integer> visited, String ans) {
    if (src == des) {
        System.out.println(ans);
        return;
    }
    visited.add(src);
    for (int nbrs : map.get(src).keySet()) {
        if (!visited.contains(nbrs)) {
            Printpath(nbrs, des, visited, ans + src);
        }
    }
}

```



1 4 5 7

	6 (4, 5)
(5, 6) (6, 6, 1, 4, 5, 7)	7 6 1, 4, 5
(4, 5, 3) (6, 6, 1, 4, 5) (7, 6, 1, 4, 5)	src=5 des=6
(5, 3) (5, 6, 1, 4)	src=4 des=6
(4, 3) (4, 6, 1)	src=1 des=6
(1, 6)	

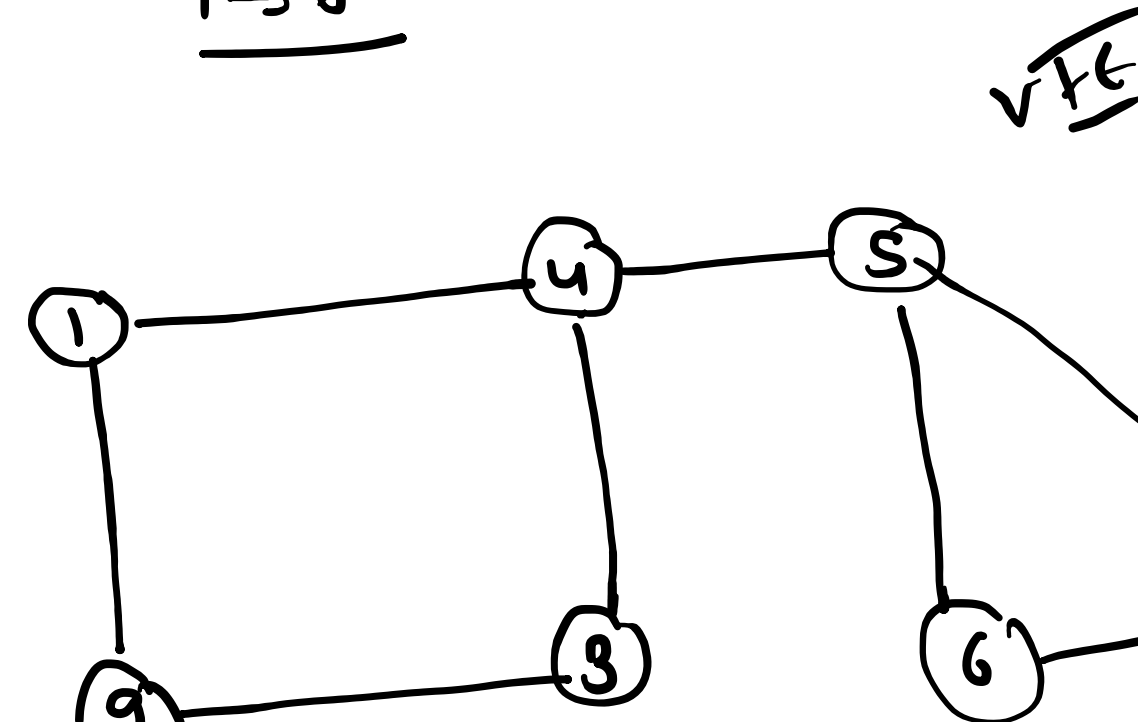
main

① B.F.S

② D.F.S

1 $\rightarrow$ 6

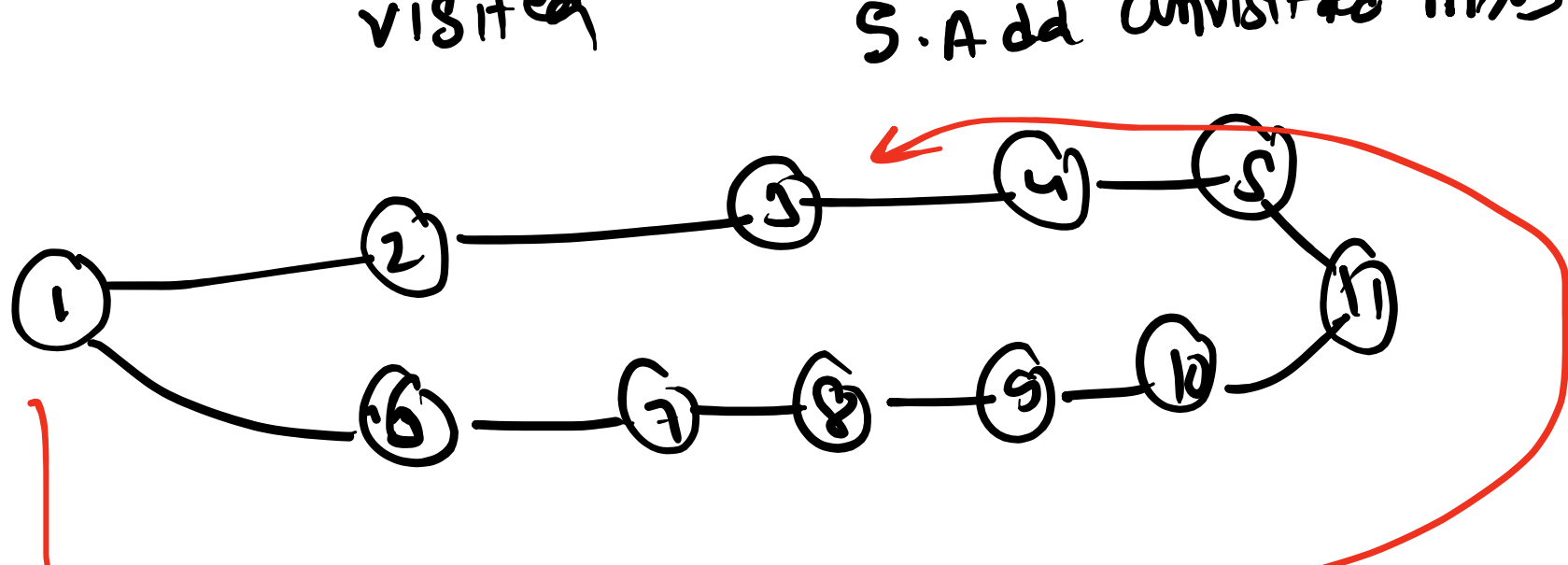
vtx



1 2 3 4 5 6 7

visited

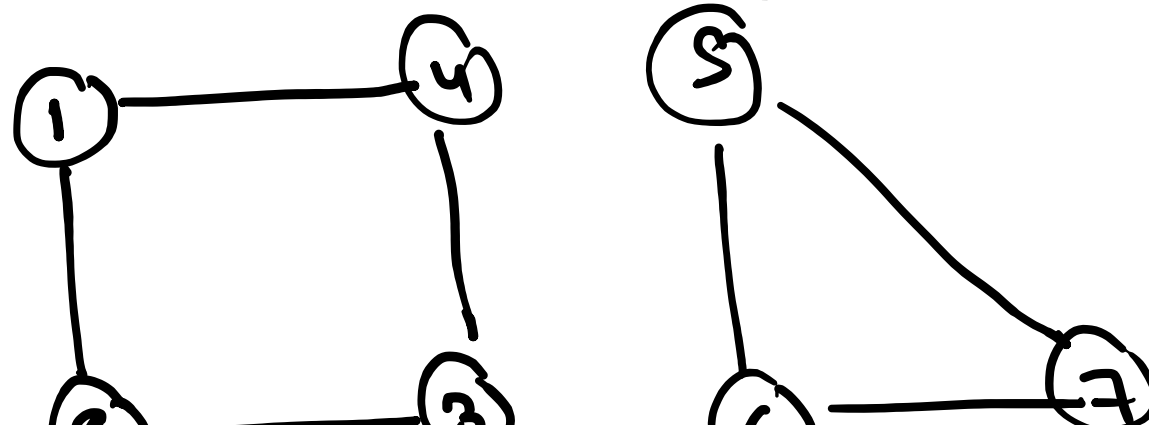
1. Remove
2. Ignore
3. mark visited
4. self work
5. Add unvisited nbrs



B.F.T

D.F.T

1 2 3 4 5 6 7 8 9

1 2 4 3
5 6 7 8

1 2 3 4 5 6 7 8 9

1. Remove

2. Ignore

3. mark visited

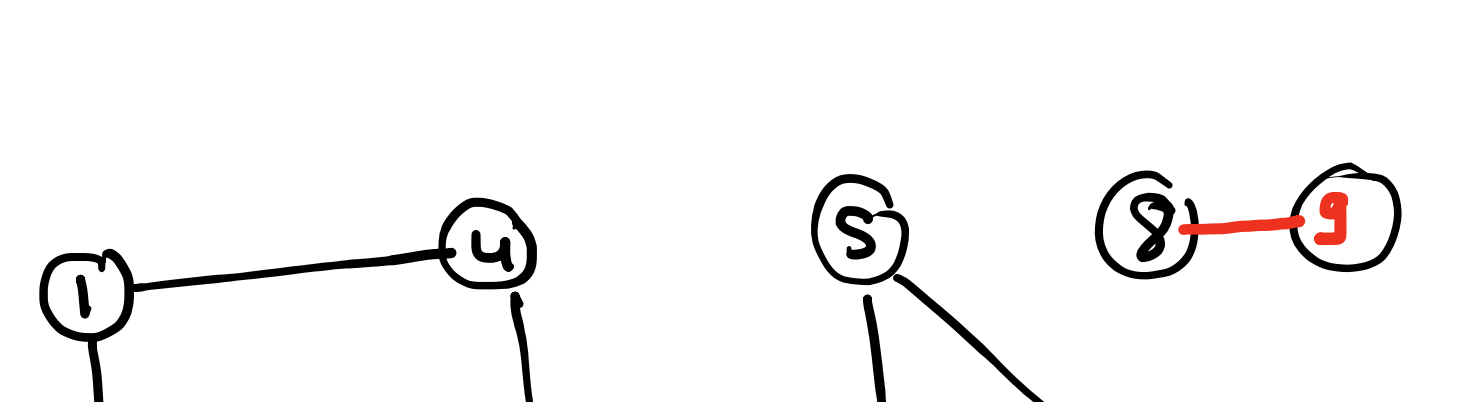
4. self work

5. Add unvisited nbrs

```

public void BFT() {
    Queue<Integer> q = new LinkedList<>();
    HashSet<Integer> visited = new HashSet<>();
    for (int src : map.keySet()) {
        if (visited.contains(src)) {
            continue;
        }
        q.add(src);
        while (!q.isEmpty()) {
            // 1. remove
            int r = q.poll();
            // 2. Ignore if Already visited
            if (visited.contains(r)) {
                continue;
            }
            // 3. marked visited
            visited.add(r);
            // 4. self work
            System.out.print(r + " ");
            // 5. Add unvisited nbrs
            for (int nbrs : map.get(r).keySet()) {
                if (!visited.contains(nbrs)) {
                    q.add(nbrs);
                }
            }
        }
    }
    System.out.println();
}

```



1 2 3 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9

Input: n = 5, edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]

Output: false

for (int i = 0; i < edges.length; i++) {

int a = edges[i][0];

int b = edges[i][1];

map.put(a, b);

0	1K
1	2K
2	3K
3	4K
4	5K
